


**Laboratorium**
**Robotika dan Sistem Cerdas**
**Departemen Teknik Komputer**
***Institut Teknologi Sepuluh Nopember***

# Laporan Akhir
# Praktikum Sistem Mikroprosesor dan Sistem Tertanam

## PENGANTAR MIKROKONTROLER

Aisyah Nuurii - 5024241008

2026

# 1 Langkah-Langkah Percobaan

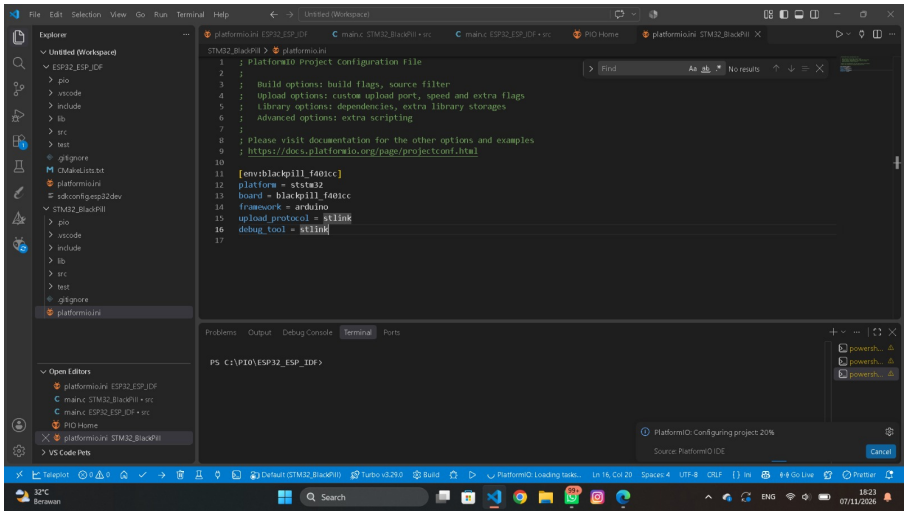
## 1.1 Percobaan 1

**Pengenalan STM32 Blackpill (Blink)**

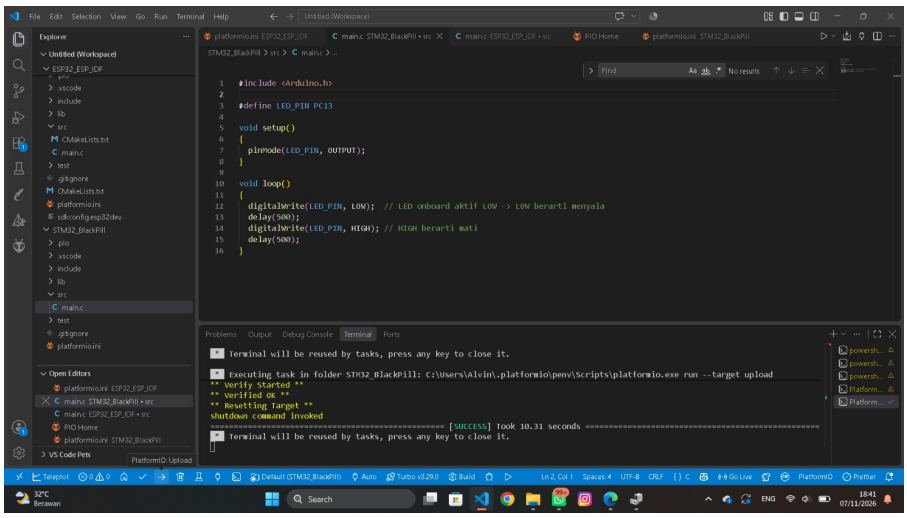
1. Menyiapkan board STM32 Blackpill, ST-Link, kabel penghubung, dan komputer yang telah terpasang Visual Studio Code serta ekstensi PlatformIO.
2. Membuka project Modul 1 yang telah dibuat sebelumnya, kemudian mengatur file `platformio.ini` menggunakan konfigurasi board `blackpill_f401cc` dengan framework Arduino.
3. Menghubungkan ST-Link ke board STM32 Blackpill dengan mencocokkan pin SWDIO, SWCLK, GND, dan 3.3V. Memastikan tegangan target ST-Link berada pada 3.3V.
4. Menghubungkan ST-Link ke komputer melalui kabel USB dan memastikan board telah terhubung dengan baik.



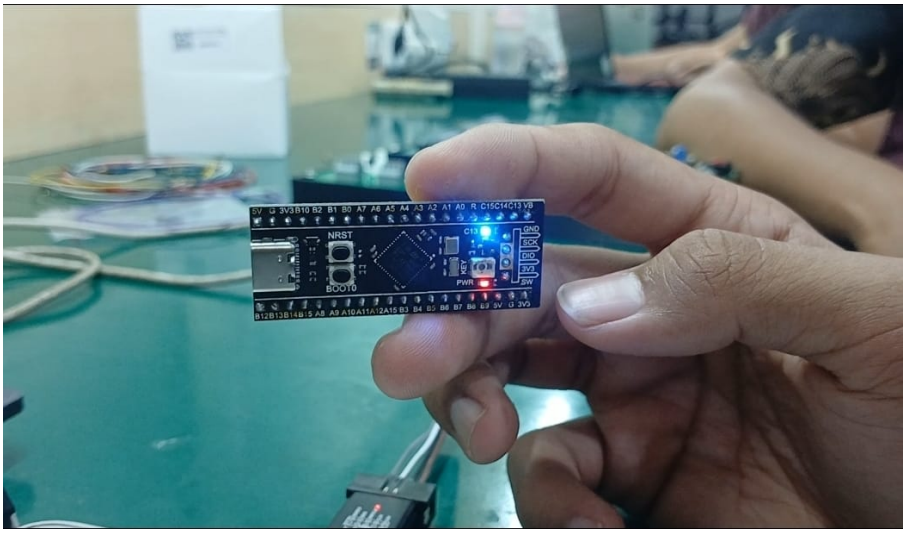

5. Membuka file `src/main.cpp`, kemudian memasukkan program Blink dengan menggunakan pin PC13 sebagai LED onboard.



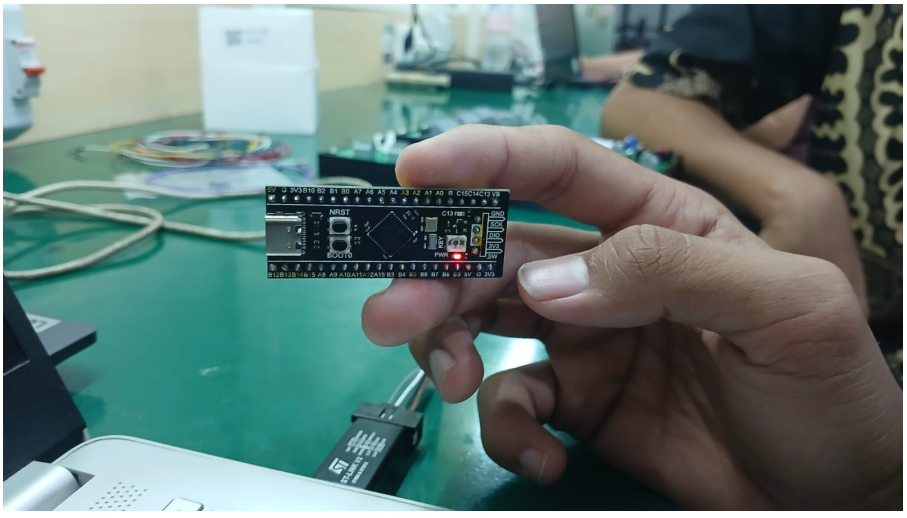

6. Menjalankan proses Build melalui PlatformIO untuk memeriksa program. Setelah proses Build berhasil, menjalankan Upload untuk mengirimkan program ke board.




7. Mengamati LED onboard pada pin PC13. Memastikan LED menyala selama 500 ms dan mati selama 500 ms secara berulang.




**Gambar 1:** Dokumentasi Percobaan Pengenalan STM32 Blackpill on



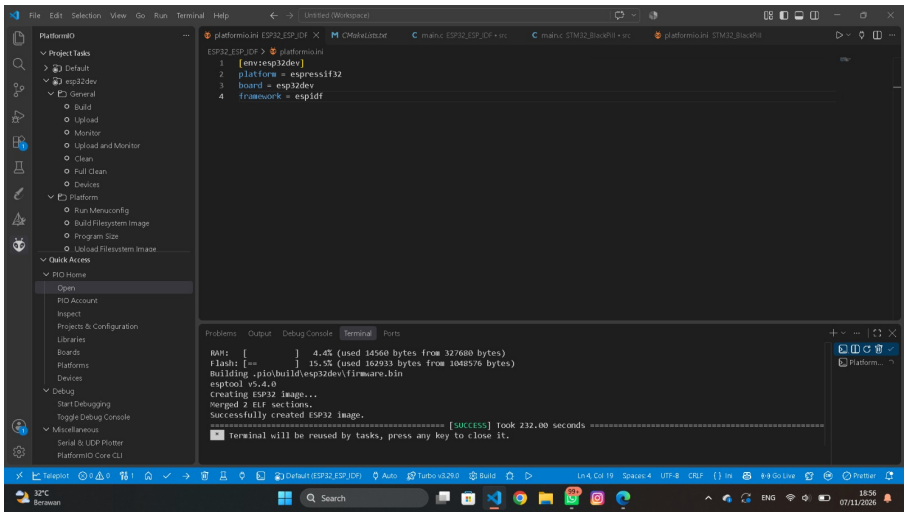

**Gambar 2:** Dokumentasi Percobaan Pengenalan STM32 Blackpill off

## 1.2 Percobaan 2

**Pengenalan ESP32 dengan Framework ESP-IDF (Blink)**

1. Menyiapkan board ESP32 DevKit, kabel USB, dan komputer yang telah terpasang Visual Studio Code serta ekstensi PlatformIO.

2. Membuka project praktikum, kemudian mengubah konfigurasi `platformio.ini` menggunakan board `esp32dev` dengan framework ESP-IDF.



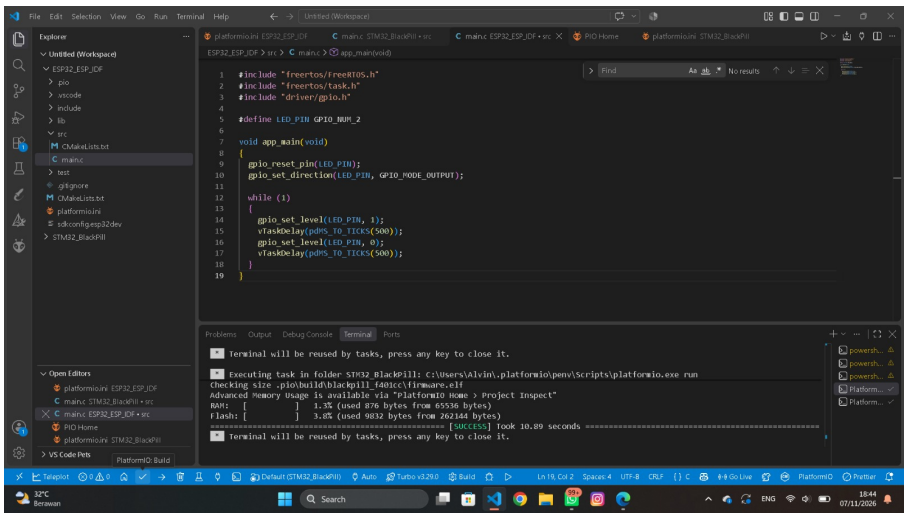

3. Menunggu proses refresh konfigurasi PlatformIO hingga selesai sebelum melanjutkan proses pemrograman.

4. Menghubungkan board ESP32 ke komputer menggunakan kabel USB dan memastikan port komunikasi board telah terdeteksi.




5. Menghapus file `src/main.cpp`, kemudian membuat file baru bernama `src/main.c`.

6. Memasukkan program Blink menggunakan framework ESP-IDF dengan GPIO 2 sebagai pin LED.

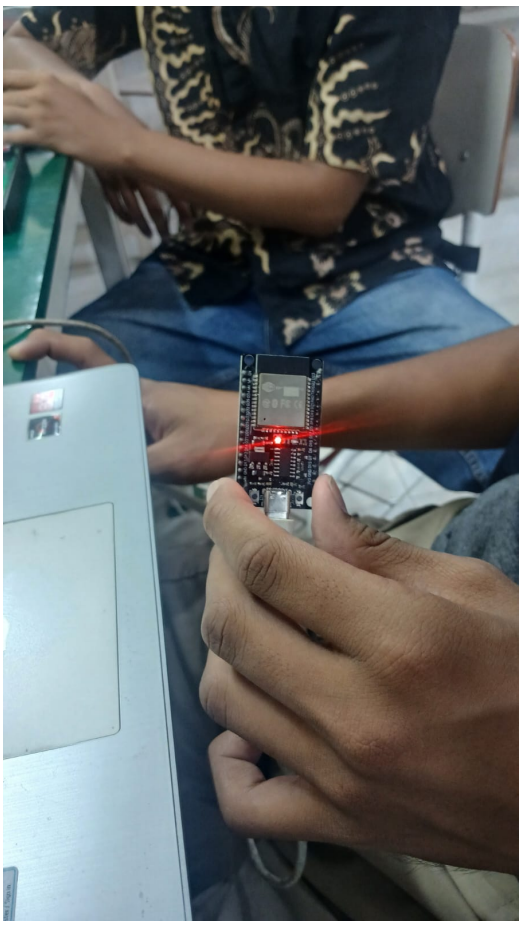



7. Menjalankan proses Build untuk memeriksa program, kemudian melakukan Upload untuk mengirimkan program ke board ESP32.

8. Mengamati LED onboard ESP32 untuk memastikan LED menyala dan mati secara bergantian dengan interval 500 ms.



**Gambar 3:** Dokumentasi Percobaan Pengenalan esp32 Blackpill on



**Gambar 4:** Dokumentasi Percobaan Pengenalan esp32 Blackpill off

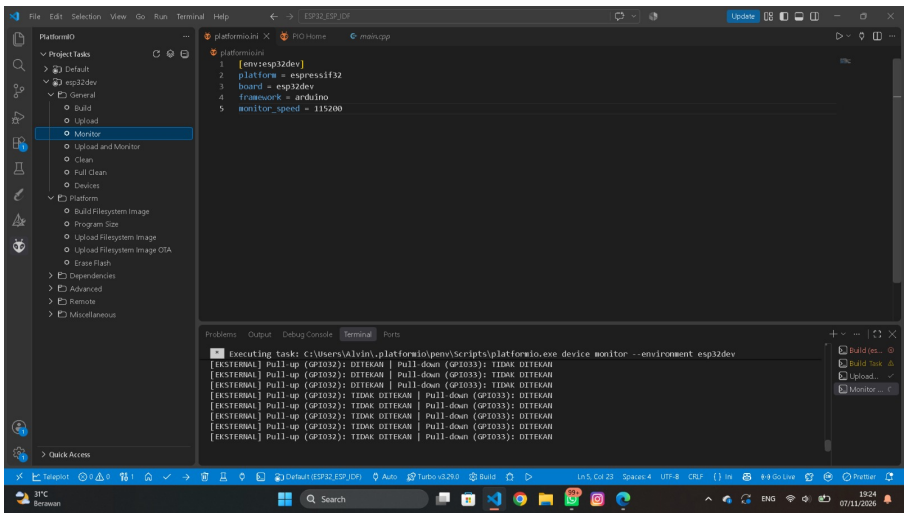
9. Mengamati struktur program dan mencatat perbedaan penggunaan fungsi GPIO pada framework ESP-IDF dibandingkan dengan framework Arduino.

## 1.3 Percobaan 3

**Pull-up dan Pull-down Eksternal serta Internal pada ESP32**

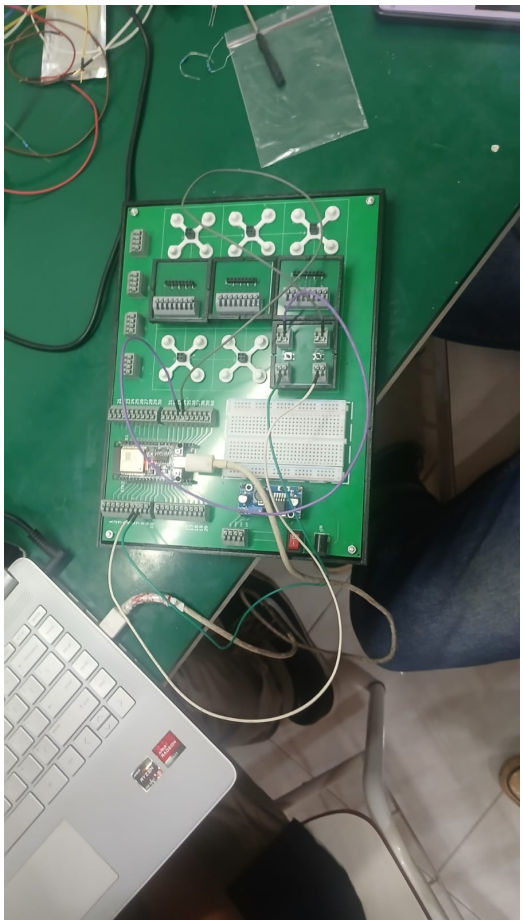
1. Menyiapkan board ESP32, dua pushbutton, dua resistor 10 k$\Omega$, breadboard, kabel jumper, dan kabel USB.

2. Mengatur konfigurasi `platformio.ini` menggunakan board `esp32dev` dengan framework Arduino. Menghapus file `src/main.c` apabila masih tersedia, kemudian membuat file `src/main.cpp`.



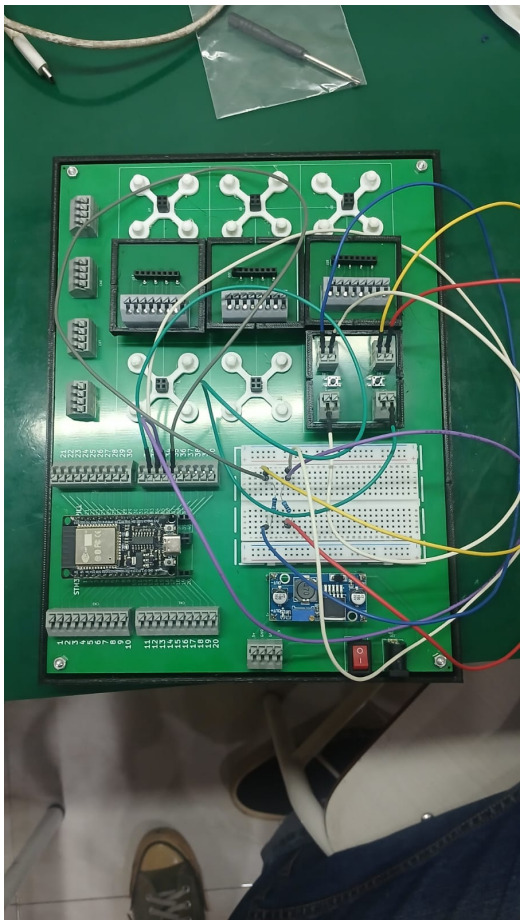

3. Merangkai pushbutton pertama pada GPIO 32 menggunakan konfigurasi pull-up eksternal. Menghubungkan salah satu kaki tombol ke GPIO 32 dan 3.3V melalui resistor 10 kΩ, sedangkan kaki lainnya dihubungkan ke GND.

4. Merangkai pushbutton kedua pada GPIO 33 menggunakan konfigurasi pull-down eksternal. Menghubungkan salah satu kaki tombol ke GPIO 33 dan GND melalui resistor 10 kΩ, sedangkan kaki lainnya dihubungkan ke 3.3V.

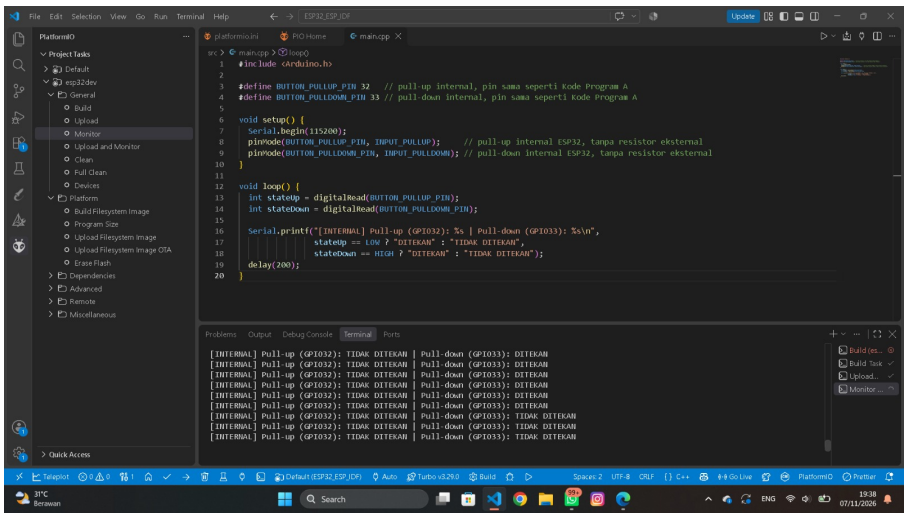


5. Memasukkan Kode Program A pada file `src/main.cpp` untuk membaca kondisi kedua pushbutton menggunakan konfigurasi eksternal.

6. Menjalankan Build dan Upload melalui PlatformIO, kemudian membuka Serial Monitor dengan baud rate 115200.
7. Menekan kedua pushbutton secara bergantian dan mengamati hasil pembacaan kondisi tombol yang ditampilkan pada Serial Monitor.
8. Melepaskan kedua resistor eksternal tanpa mengubah posisi pushbutton. Mengatur kembali rangkaian sesuai konfigurasi pull-up dan pull-down internal ESP32.
9. Memastikan pushbutton pertama tetap terhubung ke GPIO 32 dan GND, sedangkan pushbutton kedua terhubung ke GPIO 33 dan 3.3V.



10. Mengganti program dengan Kode Program B menggunakan konfigurasi `INPUT_PULLUP` dan `INPUT_PULLDOWN`, kemudian menjalankan Build dan Upload.
11. Menekan kedua pushbutton secara bergantian dan mencatat hasil pembacaan pada Serial Monitor.

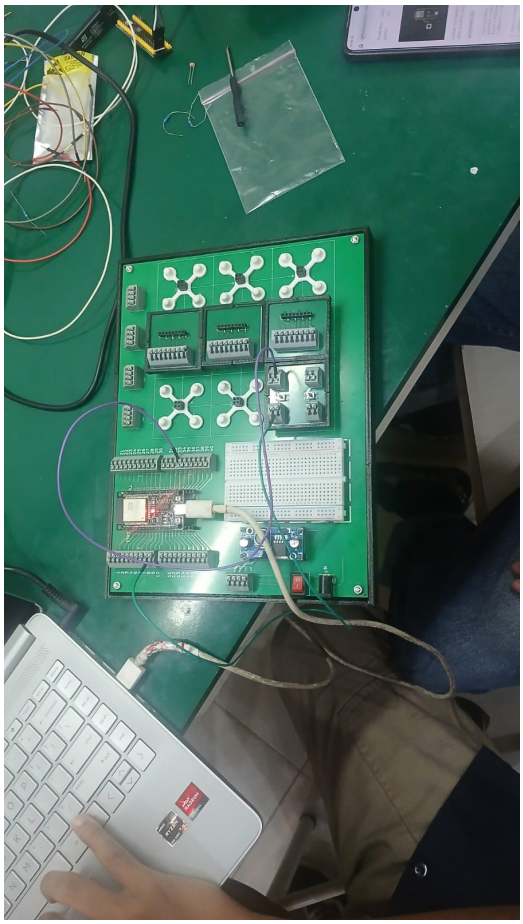



12. Membandingkan hasil pengujian konfigurasi eksternal dan internal untuk mengetahui perbedaan pembacaan kondisi tombol.

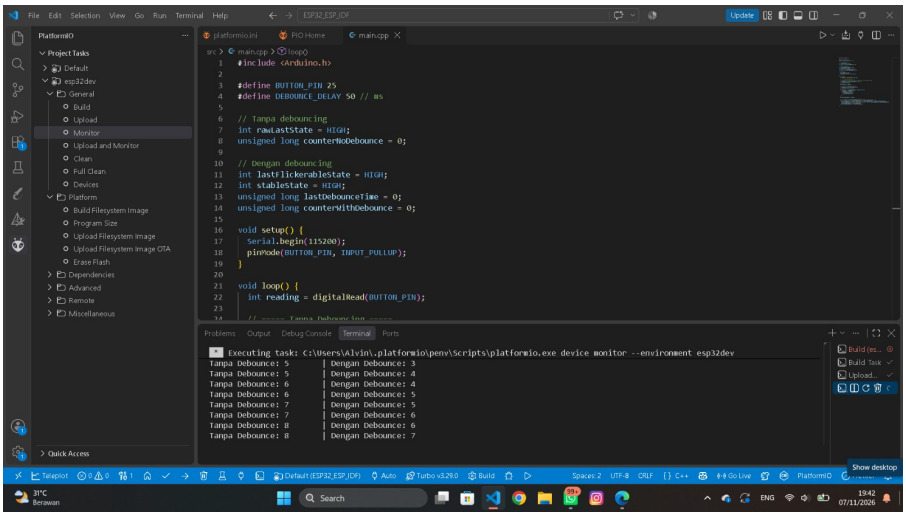
## 1.4 Percobaan 4

### Debouncing pada Input Tombol GPIO ESP32

1. Menyiapkan board ESP32, satu pushbutton, breadboard, kabel jumper, dan kabel USB.
2. Menghubungkan salah satu kaki pushbutton ke GPIO 25 ESP32 dan kaki lainnya ke GND.



3. Memastikan konfigurasi `platformio.ini` menggunakan board `esp32dev` dengan framework Arduino.
4. Membuka file `src/main.cpp`, kemudian memasukkan program debouncing yang memiliki dua penghitung, yaitu `counterNoDebounce` dan `counterWithDebounce`.
5. Mengatur nilai awal `DEBOUNCE_DELAY` sebesar 50 ms, kemudian menjalankan proses Build dan Upload.
6. Membuka Serial Monitor dengan baud rate 115200, kemudian menekan pushbutton satu kali untuk mengamati perubahan nilai kedua penghitung.
7. Mengulangi pengujian dengan menekan tombol secara cepat dan lambat. Mencatat perbedaan hasil pembacaan antara penghitung tanpa debouncing dan dengan debouncing.
8. Mengubah nilai `DEBOUNCE_DELAY` menjadi 10 ms, kemudian melakukan Build dan Upload. Mengamati perubahan hasil pembacaan setelah tombol ditekan.
9. Mengubah nilai `DEBOUNCE_DELAY` menjadi 200 ms, kemudian melakukan Build dan Upload. Mengulangi pengujian dengan menekan tombol secara cepat dan berulang.
10. Membandingkan hasil pengujian menggunakan waktu debounce 10 ms, 50 ms, dan 200 ms berdasarkan kestabilan pembacaan serta kecepatan respons tombol.



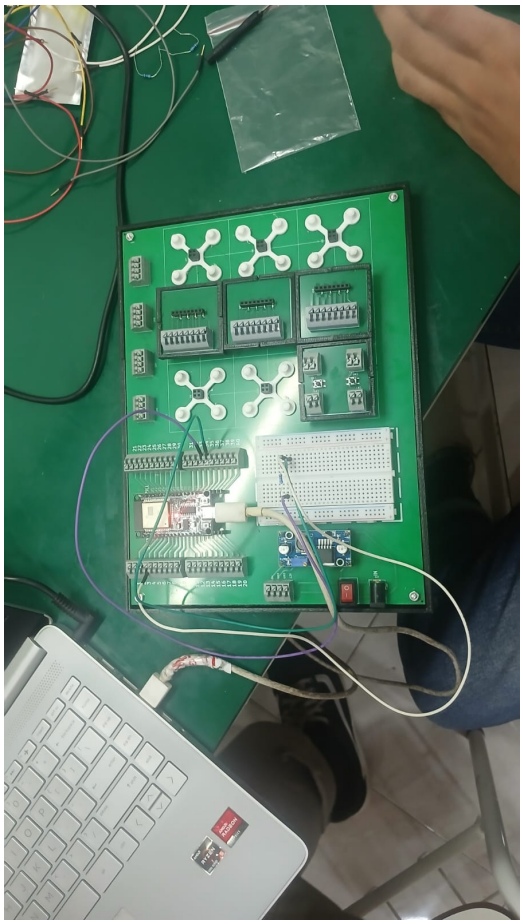

**Gambar 5:** Dokumentasi Percobaan Debouncing pada ESP32

## 1.5 Percobaan 5

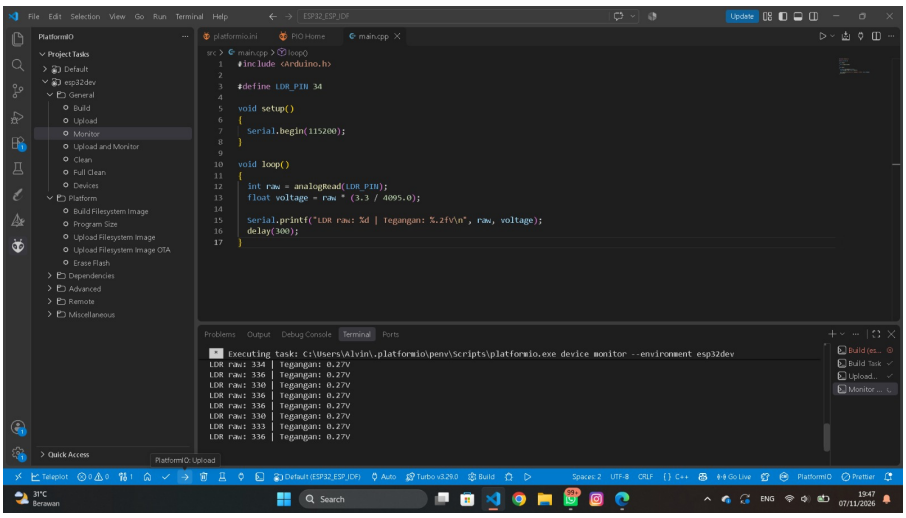
### Pembacaan ADC Menggunakan Sensor LDR pada ESP32

1. Menyiapkan board ESP32, sensor LDR, resistor 10 kΩ, breadboard, kabel jumper, dan kabel USB.
2. Merangkai sensor LDR sebagai pembagi tegangan. Menghubungkan salah satu kaki LDR ke sumber tegangan 3.3V, sedangkan kaki lainnya dihubungkan ke GPIO 34 dan resistor 10 kΩ yang terhubung ke GND.

3. Memastikan titik tengah antara LDR dan resistor terhubung ke GPIO 34 sebagai masukan analog ESP32.



4. Membuka konfigurasi `platformio.ini` dan memastikan board menggunakan `esp32dev` dengan framework Arduino.
5. Membuka file `src/main.cpp`, kemudian memasukkan program pembacaan ADC untuk memperoleh nilai analog mentah dan hasil konversi tegangan.
6. Menjalankan proses Build dan Upload melalui PlatformIO untuk mengirimkan program ke board ESP32.
7. Membuka Serial Monitor dengan baud rate 115200 untuk mengamati nilai ADC dan tegangan yang ditampilkan.
8. Menutup permukaan sensor LDR menggunakan tangan untuk mengurangi intensitas cahaya yang diterima. Mengamati perubahan nilai ADC dan tegangan pada Serial Monitor.
9. Mengarahkan cahaya lampu ke permukaan LDR untuk meningkatkan intensitas cahaya yang diterima. Mengamati kembali perubahan nilai pembacaan ADC dan tegangan.
10. Mencatat hasil pengujian pada kondisi gelap dan terang, kemudian membandingkan perubahan nilai ADC berdasarkan intensitas cahaya yang diterima sensor.




**Gambar 6:** Dokumentasi Percobaan Pembacaan ADC Menggunakan LDR

## 2 Analisis Hasil Percobaan

Percobaan pertama ini bertujuan mengenalkan board STM32 Blackpill dan menjalankan program Blink menggunakan ST-Link dengan framework Arduino. Secara teori, LED onboard di pin PC13 seharusnya berkedip setiap 500 milidetik karena logika aktif LOW, di mana nilai LOW menyalakan LED dan HIGH mematikannya. Hasil praktikum menunjukkan keluaran yang persis sama dengan teori tersebut. LED berhasil menyala dan mati secara bergantian sesuai interval waktu yang ditentukan. Hal ini membuktikan bahwa konfigurasi ‘platformio.ini‘, penyambungan kabel (SWDIO, SWCLK, GND, 3.3V), serta penggunaan fungsi dasar Arduino seperti ‘pinMode‘ dan ‘digitalWrite‘ sudah dilakukan dengan tepat tanpa perlu pengaturan register HAL yang panjang. Selama pelaksanaan, kelompok saya menggunakan file .c sebagai pengganti ‘main.cpp‘. Meskipun format filenya berbeda dari petunjuk, program tetap berhasil diproses dan diunggah ke board tanpa masalah. Kondisi ini bisa terjadi karena library ‘<Arduino.h>‘ tetap disertakan, sehingga fungsi dasar Arduino masih bisa dipanggil di dalam file C, dan sistem pemrosesan kode di PlatformIO mampu menangani hal tersebut dengan baik. Tidak ditemukan kesalahan alat atau langkah percobaan yang menghambat proses, sehingga percobaan ini bisa dinyatakan berhasil dan pemahaman mengenai arsitektur board serta cara flashing program sudah tercapai dengan baik.

Percobaan kedua ini bertujuan mengenalkan arsitektur ESP32 dan struktur program ESP-IDF untuk menyalakan LED onboard di GPIO 2. Secara teori, program menggunakan fungsi ‘gpio set direction‘ dan ‘gpio set level‘ yang dikombinasikan dengan ‘vTaskDelay‘ dari FreeRTOS untuk membuat LED berkedip setiap 500 milidetik tanpa memblokir sistem. Hasil praktikum menunjukkan keluaran yang sangat sesuai dengan teori, di mana LED berhasil berkedip dengan ritme yang stabil dan terasa lebih lancar dibandingkan percobaan sebelumnya. Penggunaan ‘app main‘ sebagai titik masuk program berjalan dengan baik, membuktikan bahwa pengaturan ‘platformio.ini‘ ke framework ‘espidf‘ dan penyambungan USB sudah dilakukan dengan tepat. Selama pelaksanaan, sempat terjadi sedikit kendala saat proses unggah program, di mana PlatformIO masih membaca konfigurasi dari percobaan sebelumnya. Masalah ini memengaruhi proses build awal, tetapi berhasil diperbaiki dengan cara menutup dan membuka ulang proyek dari awal agar sistem memuat ulang konfigurasi ‘platformio.ini‘ yang baru secara sempurna. Tidak ditemukan kesalahan pada alat atau komponen hardware, karena setelah konfigurasi diperbarui, proses build dan upload berjalan mulus tanpa hambatan. Kejadian ini

menegaskan pentingnya memastikan sistem benar-benar selesai melakukan refresh indeks sebelum menjalankan perintah build, sehingga pemahaman tentang perbedaan alur kerja antara framework Arduino dan ESP-IDF bisa didapatkan dengan baik.

Dari kedua percobaan ini, perbedaan cara memprogram dengan Arduino dan ESP-IDF terlihat jelas. Framework Arduino menggunakan fungsi sederhana seperti delay() yang menghentikan sementara seluruh proses program, sehingga lebih mudah dipahami. Sebaliknya, ESP-IDF menggunakan sistem FreeRTOS dengan fungsi vTaskDelay, di mana penundaan waktu tidak menghentikan proses lain yang sedang berjalan. Selain itu, ESP-IDF menggunakan app_main sebagai titik awal program, berbeda dengan setup dan loop pada Arduino. Meskipun penulisan kode di ESP-IDF sedikit lebih panjang dan memerlukan pengaturan yang lebih rinci, pendekatan ini membuat sistem bekerja lebih efisien.

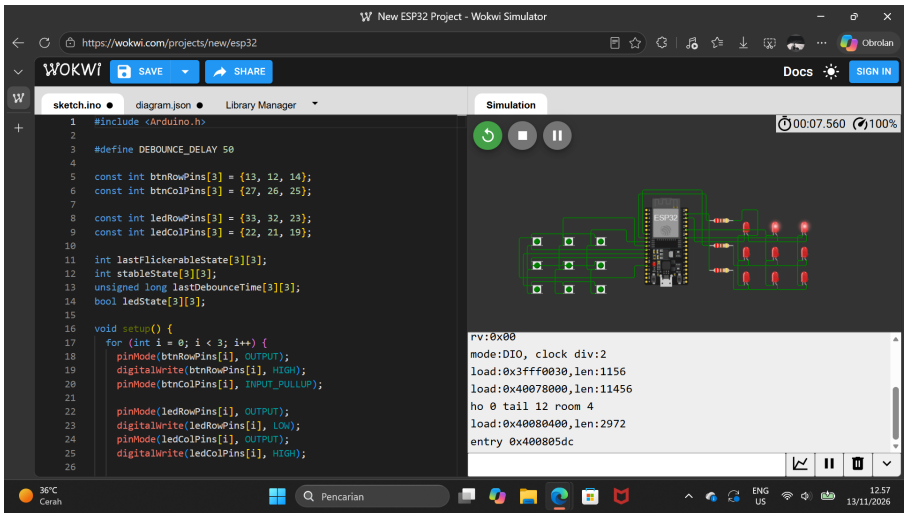
Percobaan ketiga ini bertujuan membandingkan pembacaan tombol dengan resistor pull-up dan pull-down, baik secara eksternal maupun internal pada ESP32. Secara teori, konfigurasi pull-up seharusnya membaca nilai HIGH saat tombol tidak ditekan dan LOW saat ditekan, sedangkan pull-down berlaku sebaliknya. Pada bagian eksternal, hasil yang diperoleh sempat tidak sesuai harapan karena terjadi kesalahan dalam menyambungkan pin di board ESP32. Akibatnya, saat tombol ditekan, respons yang muncul di serial monitor terasa tertunda atau tidak langsung berubah, meskipun logika program yang ditulis sudah sepenuhnya benar. Faktor yang memengaruhi hasil ini murni berasal dari ketidaktelitian dalam penyambungan kabel hardware, bukan karena kerusakan alat atau kesalahan logika kode. Berbeda dengan bagian sebelumnya, percobaan menggunakan fitur internal berjalan dengan sangat lancar. Setelah resistor eksternal dilepas dan pengaturan pin pada kode diubah menjadi ‘INPUT PULLUP‘ serta ‘INPUT PULLDOWN‘, tombol langsung merespons dengan tepat sesuai teori tanpa ada masalah keterlambatan baca. Perbandingan ini membuktikan bahwa memanfaatkan fitur internal pada ESP32 sangat membantu menyederhanakan rangkaian fisik tanpa mengurangi keakuratan pembacaan. Secara keseluruhan, praktikum ini bisa dievaluasi sebagai kegiatan yang berhasil, karena selain memastikan pemahaman tentang logika tombol, kejadian ini juga memberikan pelajaran penting untuk selalu memeriksa kembali koneksi fisik sebelum menjalankan program agar hasil yang didapat benar-benar optimal.

Percobaan keempat ini bertujuan menguji efek bouncing pada tombol mekanik dan cara mengatasinya menggunakan metode debouncing berbasis perangkat lunak di ESP32. Secara teori, tombol mekanik menghasilkan getaran sinyal saat ditekan, sehingga pembacaan tanpa penyaring bisa mencatat lebih dari satu hitungan untuk satu kali tekanan. Hasil praktikum menunjukkan kondisi yang sama, di mana nilai ‘counterNoDebounce‘ sering kali bertambah lebih dari satu saat tombol ditekan, sementara ‘counterWithDebounce‘ tetap bertambah tepat satu kali. Penggunaan resistor pull-up internal pada pin GPIO 25 dan logika pengecekan waktu menggunakan ‘millis()‘ berhasil menyaring sinyal yang tidak stabil, membuktikan bahwa kode yang ditulis sudah berjalan sesuai dengan prinsip kerja debouncing. Selama percobaan, perubahan nilai ‘DEBOUNCE DELAY‘ memberikan dampak yang jelas terhadap respons tombol. Saat nilai diatur ke 10 ms, penyaringan getaran masih terkadang kurang maksimal, sedangkan pada 200 ms tombol terasa lebih lambat merespons tekanan berulang, meskipun hitungannya tetap akurat. Tidak ditemukan kesalahan alat atau kerusakan hardware, karena perbedaan hasil murni disebabkan oleh logika program dan karakteristik fisik tombol itu sendiri. Secara keseluruhan, percobaan ini berjalan dengan lancar dan berhasil mengevaluasi pemahaman mengenai pentingnya debouncing dalam sistem input, serta menunjukkan hubungan antara kestabilan pembacaan dan kecepatan respons sistem.

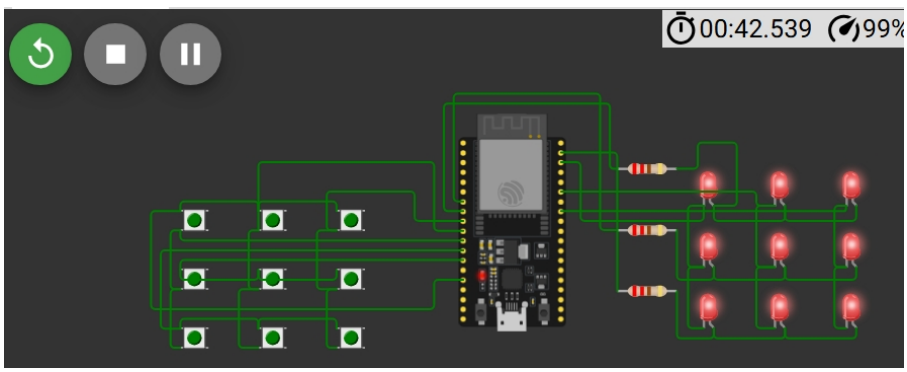
Percobaan kelima ini bertujuan mengenalkan cara membaca sinyal analog menggunakan ADC (Analog to Digital Converter) pada ESP32 dengan bantuan sensor LDR. Secara teori, rangkaian pembagi tegangan akan mengubah perubahan cahaya menjadi perubahan nilai tegangan yang dibaca oleh pin GPIO 34 dalam rentang 0 hingga 4095. Hasil praktikum menunjukkan nilai mentah dan perkiraan tegangan berubah secara dinamis sesuai kondisi cahaya di sekitar sensor. Saat LDR ditutup tangan, nilai ADC turun drastis, dan saat disinari lampu, nilainya naik mendekati batas maksimal. Perubahan ini terjadi karena resistansi LDR memang berubah sesuai intensitas cahaya, membuktikan bahwa rangkaian sudah tersambung dengan benar ke pin 34 tanpa ada kesalahan penyambungan kabel atau kerusakan komponen. Berbeda dengan percobaan tombol sebelumnya yang hanya memiliki dua kondisi pasti, pembacaan analog memberikan ribuan kemungkinan nilai di antaranya. Hal ini memungkinkan sistem mendeteksi perubahan lingkungan secara lebih halus dan bertahap, bukan sekadar kondisi nyala atau mati. Proses build dan upload berjalan lancar tanpa kendala konfigurasi, dan Serial Monitor menampilkan data yang konsisten setiap 300 milidetik.

# 3 Hasil Tugas Modul

1. Rangkaian dan code




2. Rangkaian




3. Code

WOKWI SAVE SHARE pushbutton 3x3

sketch.ino diagram.json Library Manager

```ino
#include <Arduino.h>

#define DEBOUNCE_DELAY 50

const int btnRowPins[3] = {13, 12, 14};
const int btnColPins[3] = {27, 26, 25};

const int ledRowPins[3] = {33, 32, 23};
const int ledColPins[3] = {22, 21, 19};

int lastFlickerableState[3][3];
int stableState[3][3];
unsigned long lastDebounceTime[3][3];
bool ledState[3][3];

void setup() {
  for (int i = 0; i < 3; i++) {
    pinMode(btnRowPins[i], OUTPUT);
    digitalWrite(btnRowPins[i], HIGH);
    pinMode(btnColPins[i], INPUT_PULLUP);

    pinMode(ledRowPins[i], OUTPUT);
    digitalWrite(ledRowPins[i], LOW);
    pinMode(ledColPins[i], OUTPUT);
    digitalWrite(ledColPins[i], HIGH);

    for (int j = 0; j < 3; j++) {
      lastFlickerableState[i][j] = HIGH;
      stableState[i][j] = HIGH;
      lastDebounceTime[i][j] = 0;
      ledState[i][j] = false;
    }
  }
}
```

```ino
void loop() {
  for (int r = 0; r < 3; r++) {
    digitalWrite(btnRowPins[r], LOW);

    for (int c = 0; c < 3; c++) {
      int reading = digitalRead(btnColPins[c]);

      if (reading != lastFlickerableState[r][c]) {
        lastDebounceTime[r][c] = millis();
      }

      if ((millis() - lastDebounceTime[r][c]) > DEBOUNCE_DELAY) {
        if (reading != stableState[r][c]) {
          stableState[r][c] = reading;
          if (stableState[r][c] == LOW) {
            ledState[r][c] = !ledState[r][c];
          }
        }
      }

      lastFlickerableState[r][c] = reading;
    }

    digitalWrite(btnRowPins[r], HIGH);
  }

  for (int r = 0; r < 3; r++) {
    digitalWrite(ledRowPins[r], HIGH);
    for (int c = 0; c < 3; c++) {
      digitalWrite(ledColPins[c], ledState[r][c] ? LOW : HIGH);
    }
    delay(2);
    for (int c = 0; c < 3; c++) {
```

```
  for (int r = 0; r < 3; r++) {
    digitalWrite(ledRowPins[r], HIGH);
    for (int c = 0; c < 3; c++) {
      digitalWrite(ledColPins[c], ledState[r][c] ? LOW : HIGH);
    }
    delay(2);
    for (int c = 0; c < 3; c++) {
      digitalWrite(ledColPins[c], HIGH);
    }
    digitalWrite(ledRowPins[r], LOW);
  }
}
```

Analisis

1. **Analisis Kebutuhan *Debouncing* pada Matrix Pushbutton**
    - Meskipun pada satu waktu hanya satu tombol yang ditekan, *debouncing* tetap mutlak diperlukan karena fenomena *bouncing* terjadi secara mekanis pada setiap tombol individu saat disentuh.
    - **Penyebab:** Saat tombol mekanik ditekan atau dilepas, lempengan logam di dalamnya mengalami getaran atau pantulan mikroskopis sebelum benar-benar stabil bersentuhan, memicu lonjakan sinyal listrik transien berupa pulsa naik-turun (HIGH-LOW) yang sangat cepat.
    - **Dampak pada Matrix:** Karena mikrokontroler memindai (*scanning*) baris dan kolom matriks dengan kecepatan tinggi, tanpa penundaan *debouncing*, getaran tersebut akan terbaca sebagai beberapa kali penekanan tombol berturut-turut (multi-klik palsu), sehingga mengacaukan status LED.
2. **Analisis Konfigurasi *Pull-up* dan Kondisi Pin Kolom**
    - **Konfigurasi:** Menggunakan *Internal Pull-up* (`INPUT_PULLUP`) pada pin kolom.
    - **Kondisi saat tidak ditekan:** Pin berada pada kondisi logika **HIGH** karena resistor menjaga jalur terhubung ke sumber tegangan ($V_{CC}$). Saat tombol ditekan dan baris bernilai LOW, tegangan ditarik ke ground sehingga pin berubah menjadi **LOW**.
    - **Mengapa tidak boleh *floating*:** Jika pin dibiarkan menggantung tanpa *pull-up* atau *pull-down*, pin bertindak sebagai antena yang menyerap interferensi *noise* elektromagnetik, menyebabkan mikrokontroler membaca nilai logika acak secara palsu.
3. **Perbedaan Mendasar Digital Input vs Analog Input (ADC) dengan LDR**
    - **Digital Input (Tombol):** Hanya mengenal dua status diskrit mutlak yaitu **HIGH (1)** atau **LOW (0)** berdasarkan batas ambang tegangan tertentu.
    - **Analog Input (ADC):** Mengukur nilai tegangan kontinu yang bervariasi (misalnya $0$V hingga $3.3$V) dan mengubahnya menjadi angka numerik diskrit.
    - **Mengapa ADC merepresentasikan lebih dari dua kondisi:** Perubahan resistansi LDR menghasilkan rentang tegangan kontinu. Dengan resolusi bit ADC (misalnya ESP32 menggunakan ADC 12-bit dengan $2^{12} = 4096$ tingkat kuantisasi), mikrokontroler dapat merepresentasikan ribuan gradasi tingkat cahaya yang berbeda, jauh melampaui batasan biner sederhana.

# 4 Kesimpulan

Berdasarkan kelima percobaan yang telah dilakukan, dapat disimpulkan bahwa board STM32 Blackpill dan ESP32 dapat diprogram menggunakan framework Arduino maupun ESP-IDF. Praktikum ini juga memberikan pemahaman mengenai penggunaan GPIO, konfigurasi pull-up dan pull-down, debouncing pada tombol, serta pembacaan nilai analog menggunakan sensor LDR. Seluruh percobaan berhasil menunjukkan bahwa pemrograman dan konfigurasi rangkaian yang tepat sangat berpengaruh terhadap kinerja mikrokontroler dalam membaca maupun menghasilkan sinyal.

# 5 Lampiran

## 5.1 Dokumentasi saat praktikum

